Checkpoint D - Architecture and Changeability

QuantRule

2026-11-12

Architecture description

`docs/course/architecture/src/03-architecture-description.md`, with code references: style, domain partitioning, six services, interfaces, constraints, and the twelve decisions.

Decisions

`docs/adr/2026-11-12/01-12`. Each has context, decision, alternatives, consequences, a revisit trigger, and a verification line. The two that are hard to reverse:

Decision	Why hard to reverse	Revisit trigger
Canonical digest definition	Invalidates every recorded comparison	A digest that misses a real artefact class
Run identity inputs	Invalidates deduplication and every run id	A new input that changes results

Quantified characteristics

Eight, in `02-architecture-characteristics.md`, each tied to a kata requirement and each with a measurable level.

Changeability evidence

Change	Ripple	What it cost	What stayed easy
Add a service (simulation)	2 ADRs, 15 documents, 9 issues	a document pass; no engine change	engines untouched
Add profiles	one column in four schemas, one header, one client store	small	no service changed its responsibility
Add a reading	one module, registry, parser, writer, one fixture	the widest ripple in the system	bounded to one crate

Cheap to change: a rule, a reading, a client screen, an endpoint behind a version. **Expensive:** the digest definition, the run identity, the service count. **Evidence limitation:** the ripple figures come from the change set, not from a measured time series; there is one data point per change.

Consistency

Kata R1-R8 -> eight characteristics -> six services -> twelve decisions -> the issue tree. One increment throughout: one rule in, one reproducible bundle out.

Likely questions

- **What does this structure make cheap?** A rule change, a new reading, a client screen, and a second worker.
- **What does it make expensive?** Changing the digest definition or the run identity, and adding a seventh service.
- **Residual risk:** the distributed stack is partially built; the determinism claim is bounded to one machine.